CodeForge AI - High-Value Features Documentation

This document describes the new high-value features added to CodeForge AI to enhance usability and productivity.

Table of Contents

1. Project Templates Library
2. Export/Import Functionality
3. Automated Backend Setup Script
4. API Key Management
5. Code Quality Metrics Dashboard
6. Agent Collaboration Features

---

1. Project Templates Library

Overview

The Project Templates Library provides pre-built project templates for quick project creation. It includes templates for various project types:

- **REST API**: FastAPI, Express, Flask
- **Web Applications**: Next.js, React, Vue
- **CLI Tools**: Python Click, Node Commander
- **Microservices**: Docker-ready with health checks
- **Data Science**: Jupyter, pandas, scikit-learn
- **Machine Learning**: MLflow tracking, model training

Built-in Templates

Template	Language	Framework	Description
FastAPI REST API	Python	FastAPI	Production-ready API with JWT auth
Express.js REST API	JavaScript	Express	Node.js API with Prisma ORM
Next.js Full-Stack	TypeScript	Next.js	Modern React app with App Router
Python CLI Tool	Python	Click	CLI with rich output
ML Project	Python	scikit-learn	MLflow tracking included
Microservice	Python	FastAPI	Docker + Kubernetes ready

API Endpoints

List Templates

```
GET /api/templates
```

Query parameters:
- `category`: Filter by category (rest_api, web_app, cli_tool, etc.)
- `language`: Filter by language (python, javascript, typescript)
- `framework`: Filter by framework (fastapi, express, nextjs)

Get Template Details

```
GET /api/templates/{template_id}
```

Create Project from Template

```
POST /api/templates/create-project
```

Request body:

```
{
  "template_id": "uuid",
  "project_name": "my-new-project",
  "project_description": "Optional description",
  "customizations": {
    "author": "Your Name"
  }
}
```

Create Custom Template

```
POST /api/templates
```

Frontend Usage

```
import { templatesAPI } from '@/lib/api-extended';

// List templates
const { templates } = await templatesAPI.list({ category: 'rest_api' });

// Create project from template
const result = await templatesAPI.createProject({
  template_id: 'template-uuid',
  project_name: 'My New API',
});
```

---

2. Export/Import Functionality

Overview

Export and import projects and agents for backup, migration, or sharing.

Export Features

- **Project Export**: ZIP file with all code, configs, and metadata
- **Agent Export**: JSON file with configurations and system prompts
- **Bulk Export**: Export multiple projects and agents at once
- **Optional includes**: History, metrics, related agents

Import Features

- **Project Import**: Import from ZIP files
- **Agent Import**: Import from JSON files
- **Validation**: Pre-import validation checks
- **Bulk Import**: Import multiple items at once

API Endpoints

Export Project

```
POST /api/export/project/{project_id}
```

Query parameters:
- `include_history`: Include chat/prompt history (default: false)
- `include_metrics`: Include quality metrics (default: false)
- `include_agents`: Include related agents (default: false)

Returns: ZIP file download

Export Agent

```
POST /api/export/agent/{agent_id}
```

Returns: JSON file download

Import Project

```
POST /api/import/project
```

Content-Type: `multipart/form-data`
- `file` : ZIP file
- `new_name` : Optional new project name

Validate Import

```
POST /api/import/validate
```

Validates the import file before importing.

Frontend Usage

```
import { exportAPI, importAPI, downloadBlob } from '@/lib/api-extended';

// Export project
const blob = await exportAPI.exportProject('project-id', {
  include_history: true,
  include_metrics: true,
});
downloadBlob(blob, 'my-project.zip');

// Import project
const result = await importAPI.importProject(file, 'New Project Name');
```

---

3. Automated Backend Setup Script

Overview

Comprehensive setup scripts that automate the entire backend installation process.

Linux/macOS Setup

```
chmod +x setup_backend.sh
./setup_backend.sh
```

Windows Setup (PowerShell)

```
.\setup_backend.ps1
```

What the Script Does

1. **Prerequisites Check**
   - Python 3.9+ verification
   - Node.js check (optional)
   - PostgreSQL availability

2. **Virtual Environment**
   - Creates Python virtual environment
   - Activates environment
   - Upgrades pip

3. **Dependencies**
   - Installs all requirements from requirements.txt
   - Installs development tools (pytest, etc.)

4. **Database Setup**
   - Creates PostgreSQL database and user
   - Enables required extensions (uuid-ossp, pg_trgm)
   - Falls back to SQLite for development if PostgreSQL unavailable

5. **Environment Configuration**
   - Creates .env file from template
   - Generates secure SECRET_KEY and ENCRYPTION_KEY
   - Sets up database URL

6. **Migrations**
   - Runs Alembic migrations
   - Creates initial database schema

7. **Data Seeding**
   - Seeds system prompts
   - Seeds project templates
   - Seeds default configurations

8. **Validation**
   - Verifies all packages installed
   - Tests database connection
   - Validates FastAPI app loads correctly

Script Options (PowerShell)

```
# Force recreate virtual environment
.\setup_backend.ps1 -Force

# Skip database setup
.\setup_backend.ps1 -SkipDB

# Verbose output
.\setup_backend.ps1 -Verbose
```

---

4. API Key Management

Overview

Secure storage and management of API keys with encryption at rest.

Features

- **Encrypted Storage**: Keys encrypted with Fernet (AES-128)
- **Key Validation**: Automatic validation when adding keys
- **Usage Tracking**: Track API calls per key
- **Rotation Support**: Key rotation reminders and history
- **Multiple Providers**: Support for Venice AI, OpenAI, GitHub, etc.

Supported Providers

- Venice AI
- OpenAI
- Anthropic
- GitHub
- HuggingFace
- Custom

API Endpoints

List API Keys

```
GET /api/api-keys
```

Query parameters:
- `provider`: Filter by provider
- `environment`: Filter by environment (production, development)

Add API Key

```
POST /api/api-keys
```

Request body:

```
{
  "name": "My OpenAI Key",
  "provider": "openai",
  "api_key": "sk-...",
  "description": "Production API key",
  "environment": "production",
  "rotation_reminder_days": 90
}
```

Get Key Usage Stats

```
GET /api/api-keys/{key_id}/usage?days=30
```

Rotate Key

```
POST /api/api-keys/{key_id}/rotate
```

Security

- Keys are encrypted using Fernet symmetric encryption
- Only key prefix is stored in plaintext for identification
- SHA256 hash stored for verification
- Decryption requires master encryption key (stored in .env)

Frontend Usage

```
import { apiKeysAPI } from '@/lib/api-extended';

// Add a key
const result = await apiKeysAPI.add({
  name: 'My API Key',
  provider: 'openai',
  api_key: 'sk-...',
});

// Get usage stats
const stats = await apiKeysAPI.getUsage(keyId, 30);

// Check keys needing rotation
const needsRotation = await apiKeysAPI.getKeysNeedingRotation();
```

---

5. Code Quality Metrics Dashboard

Overview

Analyze code quality with comprehensive metrics including complexity, duplication, and security.

Metrics Collected

Metric	Description
Lines of Code	Total, code, comment, blank lines
Language Distribution	Breakdown by language
Cyclomatic Complexity	Average and maximum
Maintainability Index	0-100 scale
Code Duplication	Percentage of duplicated code
Security Score	Based on vulnerability patterns
Overall Score	Combined quality score (A+ to F)

Security Scanning

Detects common security issues:
- Hardcoded passwords/API keys
- eval() and exec() usage
- Shell injection risks
- Insecure deserialization
- XSS vulnerabilities

API Endpoints

Analyze Project

```
POST /api/quality/analyze/{project_id}
```

Get Metrics

```
GET /api/quality/metrics/{project_id}
```

Get History

```
GET /api/quality/history/{project_id}?limit=30
```

Generate Report

```
GET /api/quality/report/{project_id}?format=markdown
```

Response Example


```
{
  "project_id": "uuid",
  "analyzed_at": "2024-01-15T10:30:00Z",
  "summary": {
    "total_files": 25,
    "total_lines": 5000,
    "overall_score": 85,
    "grade": "B+"
  },
  "complexity": {
    "average": 5.2,
    "max": 15,
    "high_complexity_files": ["services/complex_service.py"]
  },
  "security": {
    "score": 90,
    "issues_found": 3,
    "critical_issues": []
  },
  "recommendations": [
    {
      "category": "complexity",
      "priority": "medium",
      "message": "Consider refactoring complex functions",
      "action": "Break down functions with complexity > 10"
    }
  ]
}
```


---

6. Agent Collaboration Features

Overview

Enable AI agents to work together on projects through message passing, shared context, and task delegation.

Features

- **Agent-to-Agent Messaging**: Direct communication between agents
- **Shared Context**: Memory sharing between agents
- **Task Delegation**: Parent agents can delegate tasks to child agents
- **Collaborative Code Review**: Agents can review each other's code
- **Collaboration Graph**: Visualize agent relationships

Message Types

- `task_delegation`: Delegating a task
- `code_review`: Requesting code review
- `context_share`: Sharing context/memory
- `query`: Asking a question
- `response`: Responding to a message

API Endpoints

Send Message

```
POST /api/collaboration/messages
```

Request body:

```
{
  "from_agent_id": "agent-1",
  "to_agent_id": "agent-2",
  "message_type": "task_delegation",
  "subject": "Implement user authentication",
  "content": "Please implement JWT authentication...",
  "priority": "high",
  "requires_response": true
}
```

Create Shared Context

```
POST /api/collaboration/context
```

Request body:

```
{
  "name": "Project Architecture",
  "context_type": "architecture",
  "data": {
    "patterns": ["MVC", "Repository"],
    "database": "PostgreSQL",
    "api_style": "REST"
  },
  "creator_agent_id": "agent-1",
  "project_id": "project-uuid",
  "participating_agents": ["agent-1", "agent-2"]
}
```

Delegate Task

```
POST /api/collaboration/delegate
```

Request body:

```
{
  "parent_agent_id": "orchestrator",
  "child_agent_id": "specialist-agent",
  "task_description": "Implement database migrations",
  "delegation_type": "full",
  "instructions": "Use Alembic for migrations..."
}
```

Request Code Review

```
POST /api/collaboration/code-review
```

Get Collaboration Graph

```
GET /api/collaboration/graph
```

Collaboration Graph Response


```
{
  "nodes": [
    {"id": "agent-1", "name": "Orchestrator", "role": "coordinator", "status": "act-
ive"},
    {"id": "agent-2", "name": "Backend Dev", "role": "developer", "status": "active"}
  ],
  "edges": [
    {"source": "agent-1", "target": "agent-2", "type": "delegation", "status": "in_pro
gress"},
    {"source": "agent-2", "target": "agent-1", "type": "message", "weight": 5}
  ],
  "stats": {
    "total_agents": 2,
    "total_connections": 2,
    "active_delegations": 1
  }
}
```

Database Models

New Tables Created

Table	Purpose
`project_templates`	Store project templates
`api_keys`	Encrypted API key storage
`api_key_usage_logs`	Detailed usage tracking
`code_quality_metrics`	Quality analysis results
`code_quality_history`	Historical metrics
`agent_messages`	Agent communication
`shared_contexts`	Shared memory between agents
`task_delegations`	Task delegation tracking
`code_review_requests`	Code review requests
`export_records`	Export history
`import_records`	Import history

Getting Started

1. Run Setup Script

```
# Linux/macOS
./setup_backend.sh

# Windows
.\setup_backend.ps1
```

2. Configure API Keys

Edit `backend/.env`:

```
VENICE_API_KEY=your_key_here
GITHUB_PERSONAL_ACCESS_TOKEN=your_token_here
```

3. Start Backend

```
cd backend
source venv/bin/activate  # or .\venv\Scripts\Activate.ps1 on Windows
uvicorn main:app --reload --port 8000
```

4. Start Frontend

```
cd frontend
npm install
npm run dev
```

5. Access Application

- Frontend: http://localhost:3000
- API Docs: http://localhost:8000/docs

Security Considerations

1. **API Keys**: Stored encrypted; never expose ENCRYPTION_KEY
2. **Database**: Use strong passwords in production
3. **CORS**: Configure allowed origins for production
4. **Rate Limiting**: Consider adding rate limiting for production
5. **Authentication**: Add user authentication for multi-tenant use

---

Troubleshooting

Common Issues

1. **Database connection failed**
   - Check PostgreSQL is running
   - Verify DATABASE_URL in .env

2. **API key validation failed**
   - Check the key is correct
   - Verify network connectivity

3. **Import failed**
   - Ensure file is a valid ZIP/JSON
   - Check export version compatibility

4. **Quality analysis slow**
   - Large codebases take longer
   - Consider analyzing specific directories

Future Enhancements

- [ ] Real-time collaboration UI
- [ ] Agent marketplace
- [ ] Custom template marketplace
- [ ] Quality trends visualization
- [ ] CI/CD integration for quality gates
- [ ] Multi-tenant support